

Mini Game Hub

Made using Bash and Python(pygame)

Submitted by

Bhanu Charan	Bharadhwaj
25B0990	25B1073

Contents

1	Introduction	2
2	External Libraries Used	2
3	Features Implemented	2
4	Code Design	3
4.1	Base Game Class	3
4.2	Tic-Tac-Toe Game	3
4.3	Othello Game	3
4.4	Connect Four Game	4
4.5	Authentication System - <code>main.sh</code>	4
4.6	Leaderboard System	5
4.7	Data Visualization code	5
5	Hurdles Faced	6
6	If given more time	6
7	Bibliography	7

1 Introduction

This project involves building a secure, multi-user game hub that integrates Bash scripting for authentication and Python(pygame) for gameplay. Basically, two players authenticate, select a game and play via a graphical interface. Results are recorded and visualized through a leaderboard system.

2 External Libraries Used

- **pygame-ce**: Graphical Interface rendering and gameplay interaction
- **numpy**: Board representation and win condition checks
- **matplotlib**: Data visualization (plots)
- **csv**: For recording game results in `history.csv`
- **datetime**: To record date and time of each game in `history.csv`
- **sys**: To access command-line arguments
- **subprocess**: Used to run Bash commands from python, such as running `leaderboard.sh` from `game.py`
- **os**: Along with **subprocess**, for accessing modules from parent directories

3 Features Implemented

- Secure authentication using SHA-256 hashing
- Game selection menu
- Supports the following games:
 - Tic-Tac-Toe (extended board)
 - Othello (Reversi)
 - Connect Four
- Leaderboard system (with multiple sorting options)
- Data visualization using charts
- Replay option after each game

4 Code Design

4.1 Base Game Class

A generic class was implemented to represent a two-player board game. Code contains an initiator function and a turn switching function. The players get assigned with player 1 or 2 status and board(using numpy) gets initialized. Turn switching function is self-explanatory

4.2 Tic-Tac-Toe Game

A Tic-Tac-Toe class is created that inherits from the base class. Pygame is used for UI of the game. Board is represented using a 10x10 NumPy array, where each cell stores the state of the position(empty,player1, or player2).

Move Validation: Move validation is straightforward, as a move is allowed only if the selected cell is empty.

Win condition: The game checks for win after every move. This is done using a helper function which counts the maximum number of consecutive identical symbols (X or O) from a given cell (itself included) in a specified direction until a different symbol is encountered. This is applied along all directions by combining counts from opposite directions. If any direction results in a consecutive count greater than or equal to 6, the player wins.

To improve visual clarity, the game highlights the winning sequence by drawing a line across the board (across the sequence of winning symbols) when a player wins. If the board is fully filled without a winner, the game is declared a draw.

4.3 Othello Game

Othello class is created that inherits from the base class. Pygame is used for UI of the game. Valid moves are shown on the board for the current player. This is done by going over all the cells of the board and applying `validate_move` function to each empty position. When a move is found to be valid, it is marked in the NumPy Board representation and displayed as circular indicator on the UI.

Move validation: The `validate_move` function checks whether placing a disc at a given position results in flipping of atleast one opponent disc by going in all 8 directions from the selected cell. A move is considered valid if there exists at least one direction where a continuous sequence of opponent discs is followed by a disc of the current player.

Flipping discs: Once a valid move is made, the `flip.discs` function is executed. Similar to validation process, it checks all eight directions from the placed position. In each direction, if a sequence of opponent discs is bounded by a disc of current player, all the discs in that sequence are updated in the NumPy board array to reflect current player's colour.

Win Condition: Win condition has two stages. After each move, a function is called which checks if all the cells are filled or not. If filled, then using Numpy's sum operation over the board, it will report the winner (or maybe a tie situation). If board is not filled, then we come out of the function and proceed to next stage where valid move availability is checked for the next player using a `has_valid_moves` function. If current player has no valid moves, their turn is skipped.

However, if both players consecutively have no valid moves, the game is ended. And then winner is evaluated by using Numpy's sum operator over the board.

4.4 Connect Four Game

Connect Four class is created which inherits from the same base class. Pygame is used for the UI of the game. Board is represented with a 7x7 NumPy array. Players select a column using mouse input, and the piece is placed in the lowest available row within that column.

Move Validation: Move validation is handled by checking whether the top-most cell of a column is empty or not. If it is not empty, the column is considered full and the move is not valid.

After a valid move, the correct row is determined by scanning upward from bottom of the column until an empty position is found. Drop animation is implemented using Pygame, where the piece visually falls from the top to its final position.

Win Condition: The win condition is evaluated after each move using a directional counting approach similar to the Tic-Tac-Toe game. A recursive helper function is used to count consecutive pieces starting from the specified cell (itself included) belonging to the same player in a specific direction. Counts from opposite directions are added, and if the total is greater than or equal to 5, the player wins.

If the board becomes completely filled without any player achieving four in a row, the game ends in a draw

4.5 Authentication System - `main.sh`

The authentication system is implemented using a Bash script that handles both login and registration (hence the function is named `LorR`) for two players before starting the game.

Login: The system prompts each player to enter a username and makes sure that the usernames are valid (non-empty and distinct). If a username already exists in the `users.tsv` file, the user is asked to enter their password. The entered password is hashed using the SHA-256 algorithm and compared with the stored hashed value in `users.tsv` to verify the credentials. If the password is incorrect, the user will make sure whether they entered the correct username and then try again.

Registration: If the username does not exist, the system provides an option to register. During registration, the user is required to enter and confirm their password. Once the passwords match, the password is hashed and stored along with the username in the `users.tsv` file.

After successful authentication of both players, their usernames are passed as command-line arguments to the `game.py` file, which then starts the game hub.

4.6 Leaderboard System

The leaderboard system is implemented using a Bash script that processes the `history.csv` file to display player performance stats for each game.

The script reads the CSV file and filters entries based on the game name (Tic-Tac-Toe, Othello, or Connect Four). For each game, it calculates the number of wins and losses for every player by counting how many times a player appears as a winner and as a loser in the csv file. Games which end up in tie are not added to the csv file so every line in the csv file has a winner and a loser.

Win/Loss Ratio: Using the computed counts, the win/loss ratio is calculated for each player. If a player has zero losses, the ratio is written as *inf*, because when we sort using a -g flag (general numeric sort) , it treats *inf* as the largest value. The results are then formatted into a structured table displaying name, wins, losses and win/loss ratio.

Sorting: The leaderboard can be sorted based on different metrics such as wins, losses, or win/loss ratio (using the sort command along with flags) . This is controlled through a command-line argument passed to the script. Seperate leaderboards are generated for each game to allow independent analysis of performance.

4.7 Data Visualization code

To provide a visual summary of game statistics, charts are generated using data from `history.csv` file.

The file is read line by line, and relevant information such as the winner and game name is extracted. A dictionary is used to count the total number of wins for each player, while another dictionary keeps track of how many times each game has been played.

Top players by Wins: The players are sorted based on their total number of wins, and the top five players are selected. A bar graph is then plotted to display the number of wins for these players, providing a clear comparison of their performance.

Game Distribution: The frequency of each game is used to generate a pie chart, showing the distribution of games played which helps in visualizing which games are played more frequently

Both charts are displayed together (sub plot) using `matplotlib`, with appropriate labels and titles for clarity.

5 Hurdles Faced

- Writing win conditions without loops
- Adding animation for falling piece in Connect Four
- Matching themes across all the games (visual consistency)
- Finding apt images for game objects and background
- Writing about Hurdles faced

6 If given more time

With additional time:

- Add more games
- Add sound effects
- Add a turn indicator to clearly display the current player's turn
- Add animation effects to Othello
- Refine Visual consistency

7 Bibliography

- <http://docs.python.org/3/tutorial/classes.html>
- <http://pyga.me>
- <http://worldtohello.org>
- <http://boardgamegeek.com/boardgame/2719/connect-four>
- <https://www.w3schools.com/python/numpy/default.asp>
- https://www.w3schools.com/python/matplotlib_intro.asp